ICS 2403 DISTRIBUTED COMPUTING AND APPLICATIONS

GROUP 6

STEPHEN MWANGI	ENE221-0100/2022
SIDNEY NGIRU	ENE221-0138/2022
EUNICE KIMANI	ENE221-0101/2022
OSCAR MWANGI	ENE221-0093/2022
MICHAEL NJUGUNA	ENE221-0096/2022

Assignment 3 - Milestones 5 & 6

Distributed 5G/6G Network Service Platform

For assignment 3, we maintained the same codebase and the same 5-node topology as Assignments 1 and 2. This assignment adds two new capabilities to every existing node: the ability to take part in a distributed transaction (Milestone 5), and a per-node lock manager for studying concurrency and deadlock (Milestone 6).

---

1. Distributed Transactions (Milestone 5)

1.1 The concept

Some operations in a telecom platform aren't really "one thing". They are several things that all have to succeed together or not happen at all for example establishing a 5G session needs two separate reservations at the same time that is a bit of capacity at an Edge node *and* a bit of capacity at the RAN node. If the Edge reservation succeeds but the RAN reservation fails, the session is broken as it thinks it has RAN capacity, but never actually got it. Distributed transactions exist to prevent exactly that "half-succeeded" situation.

1.2 How it works here: Two-Phase Commit (2PC)

We implemented the standard protocol that is; **Prepare → Commit → Completion**:

1. **PREPARE**: the coordinator asks every participant node, "if I asked you to reserve this, could you?" Each node checks its own resource ledger (without actually reserving anything yet) and votes `commit` or `abort`.
2. **DECISION**: if *every* participant voted `commit`, the coordinator decides `commit`. If even one voted `abort` or didn't answer at all, the coordinator decides `abort` for everyone.
3. **COMPLETION**: the coordinator tells every participant the final decision. If `commit`, each node now actually applies the reservation (this is the only point where real resources change). If `abort`, nothing changes anywhere.

This gives us two of the four ACID properties immediately: **Atomicity** - all-or-nothing (no half-applied sessions) and **Consistency** (the platform never ends up in a state that shouldn't be possible). The other two come from things we already built: **Isolation** from the thread-safe lock inside each node's `ResourceManager` (Milestone 1), and **Durability** approximated by writing every transaction's outcome to disk.

We also added a **teardown step** (release) after a successful commit, so a session's resources are freed when it ends, otherwise every successful transaction would permanently eat into capacity and the platform would exhaust itself after a handful of sessions.

1.3 Experiment: normal operation

`milestone5_transactions.py`, Phase A - 30 session-establishment transactions, all 5 nodes healthy:

Metric	Result
Transactions committed	30 / 30
Mean transaction latency	~0.9 ms
Throughput	~700–900 tx/sec

1.4 Experiment: node failure + recovery

Phase B - `edge-2` is deliberately killed mid-run, then 20 more transactions are attempted, split evenly between ones that need the dead node and ones that don't:

Group	Result
Transactions touching the DEAD node (edge-2)	10/10 correctly **aborted**
Transactions touching only HEALTHY nodes	10/10 still **committed**
Availability for unaffected traffic during the outage	**100%**

It was ascertained that losing one node didn't corrupt anything as every transaction that touched it cleanly failed rather than applied and it did not take down the rest of the platform either. Traffic that didn't need the dead node kept working perfectly.

1.5 A genuine limitation we found (and what it means)

10 of the Phase B transactions show an acknowledgment gap. The coordinator sent an `abort` message to the dead `edge-2` node but never got a reply back for the reason that the node wasn't running.

This is a limitation of Two-Phase Commit called the "blocking problem": a coordinator can never be 100% sure a crashed participant actually received and applied its final instruction. Three-Phase Commit (3PC) is an alternative that adds an extra round specifically to reduce this uncertainty. We chose 2PC for this assignment because it's simpler to reason about and matches the Prepare → Commit → Completion model.

---

2. Concurrency and Deadlock Management (Milestone 6)

2.1 Three concepts

- **Race condition**: when two things try to read-then-update the same piece of shared data at the same time and one of the updates gets silently lost because they overlapped.
- **Mutual exclusion**: the fix is introducing a rule that says only one thing may touch this piece of data at a time and is enforced with a lock.
- **Deadlock**: when two things are each waiting for something the *other* one is holding, so neither can ever proceed. Classic example: two people each holding one chopstick, both waiting for the other's chopstick.

2.2 Race condition, demonstrated directly

`milestone6_concurrency_deadlock.py` runs 8 threads, each incrementing a shared counter 20 times (expected final total: 160):

Version	Final value	What happened
Without a lock	**20**	140 updates silently lost - threads overwrote each other
With a lock (mutual exclusion)	**160**	Every single update counted

2.3 Resource-allocation graph

We were asked to build a model using `P_i -> R_j` that simulates a process waiting for a resource. Two graphs were built:

Baseline (no contention) - 3 processes, 3 different resources, nobody waits for anybody:

```
P1 -> cpu_pool         (granted immediately)
P2 -> memory_pool       (granted immediately)
P3 -> bandwidth_pool    (granted immediately)
```

Deadlock scenario - Tx-A and Tx-B each need locks on **two different nodes** (edge-1 and core-1), acquired in **opposite order**:

```
Tx-A holds edge-1's lock, WAITS for core-1's lock (held by Tx-B)
Tx-B holds core-1's lock, WAITS for edge-1's lock (held by Tx-A)
```

Drawn as a wait-for graph, this is a 2-node cycle: `Tx-A → Tx-B → Tx-A` results in a deadlock.

2.4 Detection and recovery, distributed

We chose **detection + recovery** over prevention or avoidance because nothing on either single node knows a deadlock exists. Each node only sees its own tiny piece that is one lock, one waiter. The deadlock is only visible if you combine information from *both* nodes.

`common/deadlock_detector.py` does exactly that:

1. **Poll** every node's lock manager: who's holding what, who's waiting for what.
2. **Merge** all of that into one system-wide wait-for graph.
3. **Search** the graph for cycles.
4. **Resolve**: for each cycle found, pick a "victim" transaction and force-abort it, releasing its lock so the rest of the cycle can proceed.

Actual run output:

Step	Result
Both transactions still blocked 1.5s in?	**Yes -** genuine deadlock confirmed
Wait-for graph found	`Tx-A → Tx-B`, `Tx-B → Tx-A` (a 2-cycle)
Victim chosen	Tx-B
Both transactions finished after resolution?	**Yes**
Final outcome	Tx-A: both locks granted, completed normally. Tx-B: force-aborted, released its held lock

2.5 Resource utilization analysis

Utilization was sampled from every node's `ResourceManager` at the end of the run (`results/milestone6_concurrency_deadlock.json`, `final_resource_utilization`). Because every lock in this milestone's experiments was released immediately after use or force-released by deadlock resolution, utilization returns to near-zero at the end. The platform doesn't accumulate stuck resource holds even after a deadlock was resolved, which is itself a useful confirmation that the recovery mechanism

cleans up properly rather than just unblocking the waiting thread and leaving a resource leak behind.

---

3. Big Data & Power BI Extension

Following the exact same framework established in Assignment 2 with the same shared modules (`common/spark_utils.py`, `com-mon/big_data_layer.py`, `common/powerbi_export.py`) applied to this assignment's concepts.

3.1 Batch layer: 100,000-row transaction dataset (Milestone 5)

`assignment3_bigdata_extension.py` generates 100,000 simulated session-establishment attempts across the same 5 node-pairs used live, under three scenarios (`normal`, `high_load`, `node_down`) and runs a Spark aggregation predicting commit/abort outcomes at scale:

Scenario	Attempts	Predicted commit rate
normal	70,122	100%
high_load	19,984	100%
node_down	9,894	**0%**

Finding: at 100,000-row scale, Spark confirms exactly what the live 30-transaction experiment in Section 1 showed with real 2PC runs. A transaction touching a down node *always* aborts, cleanly, 100% of the time, never partially. The big-data layer doesn't just repeat this finding, it confirms it holds at a scale far beyond what we could afford to run live, which is the actual point of pairing a batch layer with a live speed layer.

3.2 Batch layer: 100,000-row lock-contention dataset (Milestone 6)

A second 100,000-row dataset simulates lock *requests* that is which node, which named resource, which transaction, which position in that transaction's acquisition order. Spark ranks which node, resource pairs see the heaviest simulated contention

A large-scale complement to the small, deliberately-engineered live deadlock in Section 2. Top contended resources from a sample run: `core-1/qos_token`, `ran-1/radio_channel`, `cloud-1/context_id`, each requested by 3,700+ distinct simulated transactions.

3.3 Speed layer (live)

- 50 real 2PC transactions run through the live platform: **50/50 committed** (healthy-platform baseline, consistent with Section 1.3).

- The exact live deadlock scenario from Section 2.4 re-run end to end: cycle `Tx-A-bd → Tx-B-bd → Tx-A-bd` detected, victim force-aborted, both transactions finished.

3.4 Serving layer: one shared, growing CSV folder

All batch and live results are written to the project-root `results/powerbi/` folder (filenames `m5_*.csv`, `m6_*.csv`) via `common/powerbi_export.py`.

---

4. Failure-Driven Engineering Log (Week 5–6)

Question	Response
What changed?	Added 2PC-based distributed transactions (Milestone 5) and a distributed lock manager + deadlock detector (Milestone 6) as new capabilities of the existing nodes.
What failed? (M5)	Phase A initially committed only 12/30 transactions and Phase B then showed 0/10 successes even for transactions that *didn't* touch the dead node looking like the failure had taken down the whole platform.
Why did it fail? (M5)	Committed transactions never released their resource reservations, so the RAN node's capacity was silently exhausted partway through Phase A. Phase B's "everything fails" result wasn't actually about the node failure at all rather was leftover resource exhaustion from Phase A contaminating the next experiment.

Question	Response
How was it fixed? (M5)	Added an explicit `tx_release` step (session teardown) run immediately after each successful commit, modeling a session's real lifecycle (establish → use → end) instead of resources accumulating forever.
What failed? (M6)	The deadlock experiment didn't deadlock at all in the first run as both transactions reported every lock "granted" immediately, with no waiting.
Why did it fail? (M6)	The two transactions were started 0.1 seconds apart, but local network round-trips complete in well under a millisecond. Tx-A finished acquiring *and releasing both* of its locks before Tx-B's thread had even sent its first request. There was never any overlap for a circular wait to form.
How was it fixed? (M6)	Each transaction now deliberately holds its first lock for 1 second before reaching for its second one, guaranteeing both transactions are mid-flight and holding their first lock at the same time which is what makes the circular wait genuinely happen, rather than just being theoretically possible.
What alternative was considered?	For M6: relying on random natural timing variance to eventually produce a deadlock across repeated runs is not viable instead forcing the overlap deterministically is more effective.

5. Reproducibility

- Same environment as Assignments 1–2: Python 3, standard library only for `common/transaction.py`, `common/lock_manager.py`, `common/deadlock_detector.py`.
- Run order: `python -m assignment3_m5_m6.milestone5_transactions` then `python -m assignment3_m5_m6.milestone6_concurrency_deadlock` (each launches its own fresh set of node processes; independent of each other and of earlier assignments' scripts).
- Timing constants that matter for reproducibility: Milestone 5's `prepare_timeout=1.5s`; Milestone 6's race-condition `time.sleep(0.001)` interleave delay and `HOLD_BEFORE_SECOND_ACQ` deadlock hold time - all defined as named constants at the top of their respective functions for easy adjustment.
- Big data extension: `pip install -r requirements.txt` (pyspark, pandas, numpy, pyarrow - same dependencies as Assignment 2, then `python -m assignment3_m5_m6.assignment3_bigdata_extension`
